Control Stage (CTRL) — Verification Plan

UVM Verification Environment | Version 0.1 (draft)

Project: ctrl_uvm_environment | Source specs: Control Stage, Cache Unit | October 2026

1. Overview

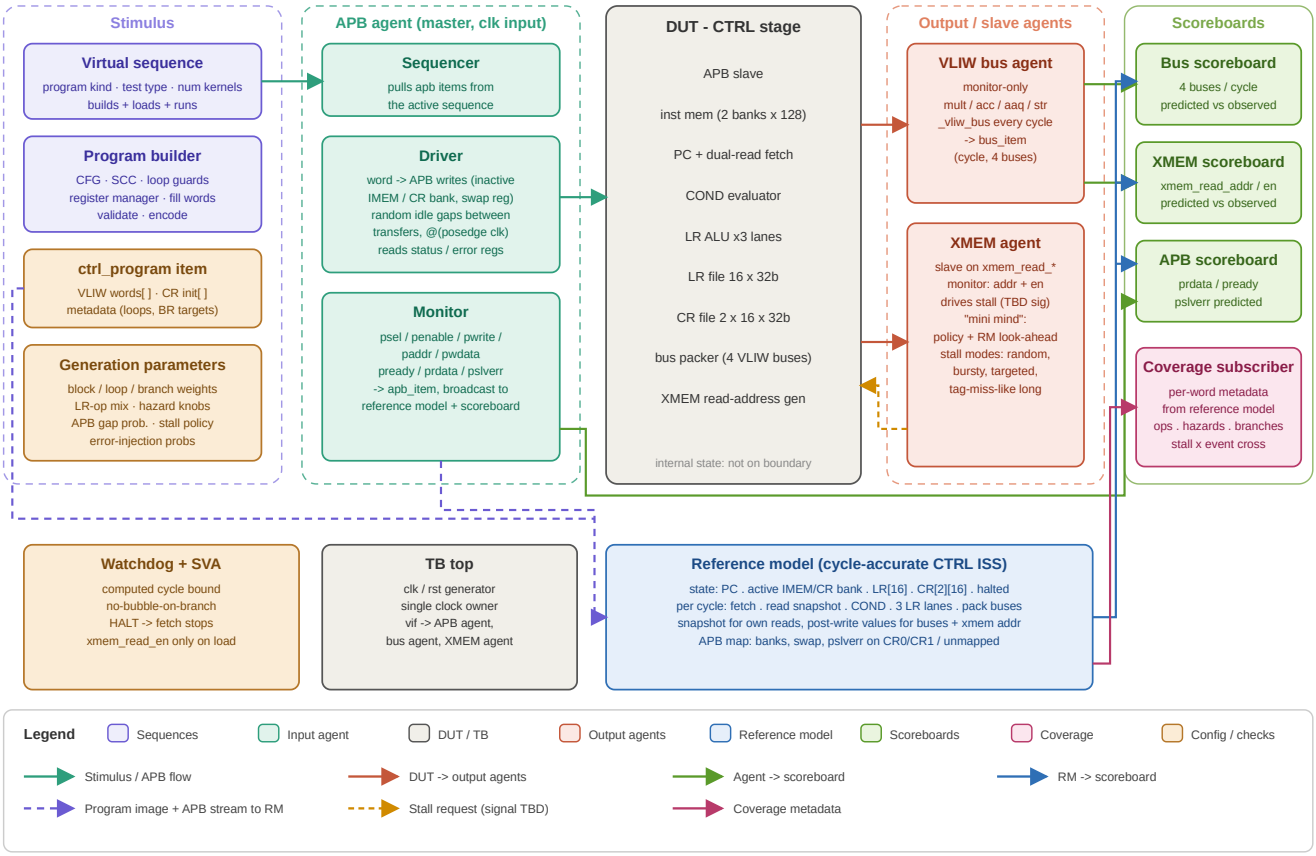
The Control (CTRL) stage is the first stage of the IPU pipeline. It owns the program counter, a double-buffered instruction memory (2 banks × 128 decoded VLIW words), the CR and LR register files, the branch resolver and three parallel LR ALU lanes. It drives four per-stage VLIW buses (`mult/acc/aaq/str_vliw_bus`) and the XMEM read address (`xmem_read_addr`, `xmem_read_en`). A RISC-V host configures it through an **APB slave** port.

This plan covers **CTRL only**. The downstream stages (MULT, ACC, AAQ, STORE) and XMEM data return are not in the environment: their buses and the XMEM address port are *monitored*, not consumed.

1.1 What makes CTRL hard to verify

- **Programs are the stimulus.** The DUT's behaviour is defined by the VLIW program in its IMEM. Random *legal and terminating* programs are the main source of coverage (§4).
- **Two read semantics in one cycle.** CTRL's own reads (LR-ALU sources, branch operands) use the *prior-cycle snapshot*; the values forwarded on the buses and used for `xmem_read_addr` are the *post-LR-write* values.
- **No-bubble branches.** Both `PC+1` and the branch target are fetched in parallel; the correct word must always be present next cycle.
- **Double-buffered configuration.** The host writes the inactive IMEM/CR bank over APB while the active bank executes.
- **Stalls.** XMEM stalls must change timing only, never results (see open question Q1, §9).

2. Environment Architecture



- The **virtual sequence** builds a random terminating program, loads it through the APB agent into the inactive bank, swaps, and lets it run. It is the only place that knows the whole program.
- The **reference model** is a cycle-accurate model of CTRL. It sees the same APB traffic (via the APB monitor) and the same program image, and predicts the four buses and the XMEM address for every cycle.
- Each **scoreboard** receives predicted items from the reference model and observed items from the relevant monitor, and compares them.

- The **coverage subscriber** receives per-word metadata from the reference model and samples the functional covergroups.
- The clock and reset are owned by the TB top. The APB agent only *receives* `clk` through its virtual interface.

3. Agents

All agents use one generic `dv_agent` pattern (as in the RX DMA environment): sequencer + driver + monitor, with `is_master` selecting driver/sequencer. Output agents are monitor-only.

3.1 APB agent (master, clock as input)

Sequencer

Pulls `apb_item`s from the active sequence and forwards them to the driver one at a time.

Driver

Drives `apb_psel / penable / pwrite / paddr / pwdata` and honours `pready` wait states. It only waits on `@(posedge vif.clk)`; it never generates the clock. The idle gap between transfers is randomised, three modes selected once per test via config_db:

- Back-to-back — no idle cycles (`gap_probability = 0`)
- Sparse random — idle cycles with probability 75%
- Deterministic — fixed pattern: transfer, idle, idle, idle

Monitor

Collects every APB transfer into an `apb_item` (address, data, direction, `prdata`, `pslverr`) and broadcasts it to the reference model and APB scoreboard. Always active.

3.2 Output agents (slave, monitor-only)

Agent	Signals monitored	Item produced
VLIW bus agent	`mult_vliw_bus, acc_vliw_bus, aaq_vliw_bus, str_vliw_bus` every cycle	`bus_item`: cycle number + four bus values
XMEM agent	`xmem_read_addr, xmem_read_en` (+ stall output, see 3.3)	`xmem_item`: address when enable is high

> **Bus format is TBD**
> The spec leaves `MULT_BUS_W / ACC_BUS_W / AAQ_BUS_W / STR_BUS_W` as "impl". The bus agent samples them as raw vectors; a `bus_layout` class (slot fields + resolved CR/LR operand fields) is needed before the scoreboard can compare field-by-field.

3.3 XMEM stall agent — the "mini mind"

The XMEM agent is a reactive slave on the XMEM read port. It owns a small **stall policy** that decides, for every XMEM access, how long to hold the controller off. It has no sequencer; its behaviour comes from the policy and a look-ahead from the reference model.

`stall_cycles = policy.decide(access)` where `access = {addr, is_branch_cycle, offset_lr_written_this_word, is_last_word_before_HALT, back_to_back_count}`

Policy mode	Behaviour	What it hunts
`NONE`	0 stall cycles	Baseline throughput
`RANDOM`	Stall with probability p each access (100 / 25 / 1-in-4, like the APB gap modes)	General handshake robustness
`BURSTY`	Quiet for N accesses, then a long stall	Restart-after-stall bugs

`TARGETED`	Stall exactly on: a load in a branch word; a load whose offset LR is written in the same word (post-write forwarding); the last word before HALT; the first word after a taken branch	The corner cases the generated program really contains
`TAG_MISS_LIKE`	One long stall of the length of a bank fill (hundreds of cycles). Mirrors the Cache Unit: CRM stalls the IPU until DMA publishes the bank tag	Long-stall tolerance, watchdog margin

The "mini mind" is: **mode + short history window + seeded RNG**, re-picked every K accesses (a "phase") so one test visits several modes. The look-ahead comes from the reference model, which already knows what the next word is. Everything is seeded, so a failing stall pattern replays exactly.

Spec gap (Q1)
The CTRL port list has **no stall / ready input**, and §9 of the spec says CTRL does no hazard interlocks. Until the stall signal into CTRL is defined, the XMEM agent can only monitor. Section 5 describes what the reference model must do once a stall exists (hold PC, hold buses, `xmem_read_en` low during bubbles).

4. Sequences and Items

4.1 ctrl_word — one VLIW word

IMEM stores **already-decoded** words, so the item mirrors the slot layout. CTRL only demultiplexes it.

Slot	Fields (randomised)	Constraints
COND (×1)	`op` ∈ {NOP, BEQ, BNE, BLT.S, BGE.S, BR, HALT}; `reg1, reg2` (LR/CR); `label` or `reg` (BR)	Branch target must be a legal word in the loaded program. `BR` uses the low 7 bits of the register. `HALT` only as a terminator.
LR (×3 lanes)	`SET, ADD, SUB, INC.MODP2, INC, DEC, ADDB, ADDBI`; `dest, src_a, src_b / imm / k`	No two lanes write the same destination; `SET` imm is signed 4-bit; `INC.MODP2` k ∈ [1,9]; `ADDB/ADDBI` dest is an even `LRDn` pair.
LOAD	NOP or `{base_idx (CR), offset_idx (LR)}`	Resulting address must fit `XMEM_ADDR_W`.
MULT / ACC / AAQ / STORE	Opaque slot payload + the CR/LR indices CTRL must resolve for that stage	CTRL does not decode the payload, only forwards it, so it is random. Indices are random over all 16 LR / 16 CR so every operand path is covered.

STORE slot
ADR 0007 removed the STORE slot, but the Control Stage page still describes `str_vliw_bus` (open question Q2). The plan keeps it as a parameter, `HAS_STORE_BUS`.

4.2 ctrl_cr_config and ctrl_program

- `ctrl_cr_config` — values for `CR2–CR14` (and `CR15`, the conventional DS half) of the inactive CR bank. `CR0 = 0` and `CR1 = 1` are hard-wired and never written, except by error injection.
- `ctrl_program` — one whole kernel: `words[0..127]`, `cr_config`, and metadata: loop headers, guard registers, `BR` target sets, expected dynamic word count.

4.3 APB sequences

Sequence	Description
`apb_load_program_seq`	Writes every word of a `ctrl_program` into the **inactive** IMEM bank, writes the CR config into the inactive CR bank, then writes the bank-swap registers.
`apb_status_seq`	Reads status/bank-ID registers while the kernel runs (must add no wait cycles to the pipeline).
`apb_error_seq`	Error injection: write `CR0` / `CR1`, write a read-only register, access an unmapped address — all expect `pslverr`.
`apb_preload_next_seq`	Loads the *next* kernel into the new inactive bank while the current one runs.

4.4 ctrl_virtual_sequence — the program-driven test controller

This is the heart of the environment. It orchestrates the whole test in five phases, using a **termination-aware program generator**: the control-flow structure is generated first, proven to terminate, and only then filled with random instructions.

Why it is not just `randomize()`

"Does this program terminate?" is a property of the control-flow graph (no unguarded cycle). A constraint solver cannot express it over a free-form array of words. So the sequence **constructs** the program stage by stage inside `body()`, and uses `randomize()` only for local choices (how many blocks, which edge, which opcode). The generation parameters are randomised once per test and distributed through config_db, as in the RX plan.

Test flow

Phase	Action
0 — Configure	Randomise `ctrl_gen_parameters` and `stall_policy` once; publish via config_db.
1 — Build program	Run the 7 generator stages below. Result: a validated `ctrl_program`.
2 — Load	`apb_load_program_seq`: program into the inactive IMEM bank, `CR2`–`CR14` into the inactive CR bank, then swap. The new active bank starts at PC 0 *(start mechanism: Q4)*.
3 — Run	While the kernel executes: optionally preload the next kernel (`apb_preload_next_seq`), read status, inject APB errors. The XMEM agent applies its stall policy.
4 — Finish	Wait for the predicted HALT retirement or the watchdog. Check scoreboards, sample coverage, drop objection.

`test_type`: **REGULAR** — one random kernel (fast). **REAL** — 10–100 kernels back-to-back, each preloaded into the inactive bank while the previous one runs, then swapped (long, high coverage value).

`program_kind`: `STRAIGHT_LINE`, `SINGLE_LOOP`, `NESTED_LOOPS`, `BRANCH_HEAVY` (branch in consecutive words), `INDIRECT_BR`, `LR_HAZARD` (same-word read-after-write), `LOAD_HEAVY`, `TIGHT_LOOP` (one-word loop), `BANK_EDGE` (program fills all 128 words, branches to word 0 and 127).

The seven generator stages

```
build_cfg() -> analyze_loops() -> insert_guards() -> alloc_registers() -> fill_words() ->
validate() -> emit()
```

#	Stage	What it does (CTRL specifics)
1	**build_cfg**	Create ENTRY (PC 0) and END (the `HALT` word). Choose a random number of blocks so the total stays within `IMEM_BANK_DEPTH = 128` words. Lay a **spine** ENTRY→B0→…→Bn→END first (HALT is always reachable), then add random forward conditional branches, **backward edges** (loops) and `BR` nodes with a pre-chosen legal target set, e.g. `{B3, B8, B12}`, with an edge to each target.
2	**analyze_loops**	SCC analysis (Tarjan) on the CFG. Every SCC with more than one node, or a self-loop, is a possible infinite run — this includes cycles created by `BR` edges. Record headers, members and nesting depth.
3	**insert_guards**	Each SCC gets a guard counter in an LR: initialised in the pre-header with `SET LRg CRn` (budget in a host-populated CR) or `SET LRg #imm` (−8..7). Each iteration executes `DEC LRg 1` and a guard branch (`BNE LRg CR0 header` / `BEQ LRg CR0 exit`) on **every** cycle path. See the snapshot rule below.
4	**alloc_registers**	Register manager (table below). Guards and `BR` target registers are chosen and protected *before* any random instruction is generated.
5	**fill_words**	Fill each block with random LR ops, a LOAD, and random MULT/ACC/AAQ/STORE payloads, touching only `FREE` registers. The COND slot of the last word is **derived from the CFG edges**, never random. Hazard knobs deliberately make a lane write an LR that the same word's bus operand / load offset reads (post-write forwarding) and its branch reads (snapshot).

6	**validate**	All branch/jump targets are legal words ≤ 127 and fit the label encoding; no protected register is written by random code; no two lanes in one word write the same register (including `LRD` pair aliasing); every cycle has a guard; HALT is reachable from every word; compute `max_dyn_words`. Failure is `uvm_fatal` — a **generator** bug, not a DUT bug.
7	**emit**	Resolve labels to word addresses, fill the `BR` target CRs *after* layout (optionally with random junk in the upper bits to test that only the low 7 bits are used), encode to decoded VLIW words, and publish the program image + metadata to the APB load sequence, the reference model, the stall agent and coverage.

Register manager

State	Registers	Rule
`FREE`	LR not used by the generator	Random instructions may read and write.
`LOOP_GUARD`	One LR per live loop	Written only by the guard's `SET` / `DEC`.
`CONTROL_FLOW`	LR holding a `BR` target	Read-only to random code. A CR target needs no protection (programs cannot write CR).
`RESERVED`	`CR0` (=0), `CR1` (=1), and any `LRD` pair that overlaps a protected LR	An `ADDB/ADDBI` on `LRDn` writes `LR(n)` and `LR(n+1)`, so both must be `FREE`.

The snapshot rule — worked loop example

An LR write in cycle N is **not** visible to CTRL's own branch read in cycle N. So a latch word `DEC LR5 1 ; BNE LR5 CR0 L` compares the value *before* the decrement:

Iteration	LR5 read by BNE	Taken?	LR5 after DEC
1	2	yes	1
2	1	yes	0
3	0	no — falls through	−1

With budget *n* the body runs *n* + *1* times. If the `DEC` is in an earlier word than the branch, it runs *n* times. The generator and the reference model must use the same rule; `validate` therefore also dry-runs the program on the reference model.

Termination and the watchdog bound

```
max_dyn_words = sum over words( 1 x product(guard_budget + 1) of the enclosing loops )
watchdog_cycles = max_dyn_words x (1 + max_stall_per_access) + margin
```

Keep `max_nest_depth` small (≈3): nested budgets multiply. A timeout is therefore either impossible (validate would have caught it) or a real DUT bug. `HALT` stops fetch, and the kernel is complete when the HALT word retires through the chain, so the end-of-kernel check is: reference model reaches HALT, no further bus changes or `xmem_read_en` after it.

4.5 Generation parameters

Parameter group	Controls
Program kind weights	Relative probability of each `program_kind`
CFG shape	Blocks per program, loop probability, back-edge probability, `BR` fan-out, max nesting depth
Guard budgets	Min/max loop budget; share held in CR vs. immediate
LR-op mix	Weight per LR opcode; lanes active per word (0–3)
Hazard knobs	Probability of same-word write→bus-operand, write→load-offset, write→branch-operand
Valid / gap probability	APB idle-gap mode (back-to-back / sparse / deterministic)
Stall policy	Mode weights, stall length ranges, phase length K
Error injection	Probability per kernel of each APB error sequence

Kernel count	Min/max kernels in a REAL test

5. Reference Model

A cycle-accurate model of CTRL (not transaction-level: the spec's behaviour is defined per cycle). It receives APB items from the APB monitor, so it tracks bank writes and swaps exactly like the DUT.

5.1 State

`PC` (7 bits), active IMEM bank, active CR bank, `LR[16]` (32 bit), `CR[2][16]` with `CR0=0`, `CR1=1`, `halted`, `imem_rd_sel`.

5.2 Per-cycle behaviour

Step	Model action
Fetch	`inst_vliw = imem[active][PC]`. Non-branch next word = `PC+1`. On a branch, both `PC+1` and the target are candidates; `taken` selects. No bubble is ever inserted for a branch.
Snapshot reads	Branch operands and LR-lane sources are read from the values committed at the end of the previous cycle.
COND	`BEQ / BNE / BLT.S / BGE.S` evaluate; `BR` is unconditional, target = low 7 bits of the register; `HALT` stops fetch (PC and word hold).
LR lanes	Three lanes compute in parallel from the snapshot (32-bit wrap; `INC.MODP2` mask; `ADDB/ADDBI` per-byte saturate [0,255]) and commit at end of cycle.
Forwarding	The four buses carry the **post-write** resolved operands. `xmem_read_addr = CR[base] + LR[offset]` from the post-write LR; `xmem_read_en = load != NOP`.
APB	Register map, inactive-bank writes, swap effect on the next fetch cycle; `pslverr` on write to `CR0/CR1`, read-only registers, or unmapped addresses; the LR file is not writable over APB.
Stall (Q1)	When defined: hold PC and word, hold buses, drive `xmem_read_en` low during the bubble.

The model also emits per-word metadata (ops, branch taken, hazard flags, loop iteration) to the coverage subscriber and a look-ahead stream to the stall agent.

6. Scoreboards

6.1 Comparison mechanism

Each scoreboard holds an expected queue (from the reference model) and an observed queue (from the monitor), compares per cycle, and reports the first mismatch with cycle, PC and word. Timing-only changes (stalls) must not change any compared value except the cycle alignment the reference model already accounts for.

6.2 Scoreboard instances

Scoreboard	Predicted vs observed
Bus scoreboard	All four VLIW buses, each cycle: slot fields and resolved CR/LR operands (post-write values)
XMEM scoreboard	`xmem_read_en` and `xmem_read_addr` each cycle
APB scoreboard	`prdata`, `pready`, `pslverr` for every transfer

6.3 Assertions (SVA, bound to the DUT)

- After a branch word, the next cycle's word is the one at `taken ? target : PC+1` (no bubble).
- After `HALT`, no new word is fetched until the next kernel starts.
- `xmem_read_en` is high only when the load slot is non-NOP.
- `CR0 == 0` and `CR1 == 1` always.

- No APB wait cycles caused by IPU activity (`pready` depends only on the slave write port).

7. Functional Coverage

7.1 Word-level coverpoints

- Each LR opcode × each lane (0, 1, 2); 0–3 lanes active in one word.
- `SET` mode (CR vs. immediate; immediate −8, −1, 0, 7); `INC.MODP2` k ∈ 1..9; `ADDB/ADDBI` saturation at 0 and 255; 32-bit wrap in `ADD/SUB/INC/DEC`.
- Each COND op, taken and not taken; branch operands LR vs. CR; `BR` source LR vs. CR; `BR` with junk upper bits.
- Load: NOP vs. active; base CR index; offset LR index.

7.2 Scenario-level coverpoints

- CFG shape: loop count, nesting depth, SCC size, one-word loop, `BR` fan-out, backward vs. forward branch.
- Branch in consecutive words; branch to `PC+1`; branch to itself; branch to word 0 and word 127; program length (1, 2, 64, 127, 128 words).
- Guard exit by guard vs. by data condition; `DEC` before vs. in the branch word.
- Back-to-back kernels; preload during run; swap immediately after HALT; APB traffic during execution.
- APB: each error type; gap mode; read during run.

7.3 Required cross coverage

- **Snapshot hazards:** LR written in word N × read by {lane source, branch operand, each bus operand, load offset} in the same word.
- **Stall × event:** {each stall mode} × {load in branch word, load after LR write, last word before HALT, first word after taken branch, back-to-back loads}.
- COND op × taken × same-word LR write to a branch operand.
- Kernel kind × stall mode × APB gap mode.

8. Key Parameters

Parameter	Default	Note
`REG_WIDTH`	32	CR/LR width (the spec's block diagram says 20 — Q3)
`IMEM_BANKS / IMEM_DEPTH / IMEM_BANK_DEPTH`	2 / 256 / 128	PC width 7 bits
`IMEM_BANK_READ_PORTS`	2	2R1W per bank: dual fetch on branch
`CR_BANKS / CR_REG_COUNT`	2 / 16	`CR0=0`, `CR1=1` hard-wired
`LR_REG_COUNT / LRD_REG_COUNT / LR_LANES`	16 / 8 / 3	LRD = register-pair aliases
`COND_OP_COUNT / LR_OP_COUNT`	6 / 8	COND: 5 branches + HALT
`APB_ADDR_W / APB_DATA_W`	32 / 32	
`XMEM_ADDR_W`	impl	Truncation of CR+LR sum: Q5
`*_BUS_W`	impl	Layout needed for scoreboard: Q6
`lock`-style test knobs	—	`test_type, program_kind, num_kernels, max_nest_depth, max_loop_guard`

9. Open Questions and Spec Issues

#	Issue	Impact on the plan

Q1	CTRL has **no stall / ready input** and §9 says it does no interlocks, yet §7.1 mentions a "bubble". How does an XMEM stall reach CTRL? (The Cache Unit stalls the IPU on a tag miss.)	Defines the XMEM agent's output signal and the reference model's stall behaviour. **Highest priority.**
Q2	ADR 0007 removed the STORE slot; the Control Stage page still has `str_vliw_bus` (action A-4).	Plan keeps `HAS_STORE_BUS` as a parameter.
Q3	Block diagram says CR/LR are 20-bit; the parameters and text say 32-bit.	Plan uses 32.
Q4	How a kernel starts and how completion is signalled is not on the port list (`kernel_done` is mentioned in prose; the bank-swap is described as both an APB register and an external `imem_bank_sel`).	Start = swap register write (assumed). End = reference model HALT + monitors, until a status register is defined.
Q5	`XMEM_ADDR_W` vs. the 32-bit `CR + LR` sum: truncate or error?	Reference model address width.
Q6	Bus layouts (`*_BUS_W`), APB address map and the IMEM word width (decoded VLIW is wider than 32 bits, so multiple APB writes per word?) are not defined.	Needed for `bus_layout` and `apb_load_program_seq`.
Q7	Spec text says "11 mnemonics" and "seven branches"; the tables list 8 LR ops and 6 COND ops (5 branches + HALT). §10.1 refers to "see §10" for the same-destination rule.	Plan follows the tables.
Q8	The Cache Unit spec is attached but out of scope. It is used only to model realistic stall lengths (`TAG_MISS_LIKE`).	None.

10. Milestones

- **M1** — APB agent with clock plumbing, directed program load, reference model for non-branch words, bus scoreboard.
- **M2** — Generator stages 1–3 and 6–7 (spine + guarded loops, LR ops only), branch model, no-bubble assertions.
- **M3** — Register manager, loads, `BR` indirect jumps, `LRD` pairs, hazard knobs, APB error injection.
- **M4** — XMEM stall agent policies (after Q1 is closed), REAL test (back-to-back kernels).
- **M5** — Coverage closure and seed regression.